E-Ticaret Satış Analizi

PostgreSQL Primary–Replica + Python ETL + Grafana | Ubuntu / Docker Compose

Uçtan uca uygulama rehberi (revize mimari) — Sıfırdan kurulum, mevcut projeyi koruyarak geçiş, SQL ve Python kodları, replikasyon doğrulaması, dashboard ve sorun giderme.

Önemli mimari not: Primary ve Replica iki ayrı PostgreSQL instance'ıdır. Host üzerinde Primary 5432, Replica 5433 portuna yayınlanır; Docker ağında her ikisi de 5432 dinler. Ayrı port tek başına yük ayrımı değildir; asıl ayrım ayrı instance ve salt okunur Replica ile sağlanır. Aynı VM içinde donanım kaynakları yine ortaktır.

1. Mimari ve çalışma sırası

```
Örnek siparişler -> PRIMARY (postgres-primary:5432)
                            |  WAL streaming replication
                            v
                       REPLICA (postgres-replica:5432)
                         |                  |
                   Python READ         Grafana READ
                         |
                  Python hesaplama
                         |
                  Python WRITE -> PRIMARY

Ubuntu host: Primary 127.0.0.1:5432 | Replica 127.0.0.1:5433
Grafana: http://localhost:3000 (VM içinden)
```

Python, orders tablosunu Replica'dan SELECT ile okur; günlük özetleri hesaplar ve Primary üzerindeki daily_sales_summary tablosuna yazar. WAL kayıtlarıyla özetler de Replica'ya gelir. Grafana yalnızca Replica'ya bağlanır.

2. Ön koşullar ve proje klasörü

Ubuntu VM'de Docker Engine ve Docker Compose v2 kurulu olmalıdır. VM için yaklaşık 4 GB RAM veya üzeri önerilir; mevcut kaynaklara göre ayarlayın. Aşağıdaki komutlar Ubuntu terminalinde çalıştırılır.

```
docker --version
sudo docker compose version
sudo systemctl is-active docker
mkdir -p ~/ecommerce-analytics/{postgres,python,grafana,backups}
cd ~/ecommerce-analytics
```

Mevcut projeden geçiyorsanız: Önce mevcut PostgreSQL'in yedeğini alın. Eski container adı ecommerce-postgres ise:

```
sudo docker exec ecommerce-postgres pg_dump \
  -U ecommerce_user -d ecommerce_db -Fc \
  > backups/ecommerce_backup.dump
ls -lh backups/ecommerce_backup.dump
sudo docker exec -i ecommerce-postgres pg_restore -l \
  < backups/ecommerce_backup.dump | head
```

Mevcut volume'u korumak için docker compose down -v kullanmayın. Compose dosyasını değiştirmeden önce eski projeyi sudo docker compose down ile durdurun. Mevcut named volume'un adı proje adına göre değişebilir; sudo docker volume ls ile kontrol edin. Aynı Compose proje klasörünü ve postgres_data volume tanımını korursanız mevcut veri kullanılabilir.

3. Dosya yapısı

```
ecommerce-analytics/
├── docker-compose.yml
├── .env
├── postgres/
│   ├── init.sql
│   └── replica-entrypoint.sh
├── python/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── grafana/
└── backups/
```

4. Ortam değişkenleri (.env)

Aşağıdaki parolalar yalnızca yerel laboratuvar örneğidir. Gerçek kullanımda güçlü ve farklı parolalar seçin; .env dosyasını Git'e eklemeyin.

```
POSTGRES_PASSWORD=ecommerce123
REPLICATION_PASSWORD=replica123_change_me
GRAFANA_ADMIN_PASSWORD=admin123_change_me

chmod 600 .env
printf '
.env
backups/
' >> .gitignore
```

5. PostgreSQL tabloları ve başlangıç verileri

postgres/init.sql dosyasına aşağıdaki içeriği yazın. Bu dosya yalnızca boş Primary veri dizininin ilk açılışında otomatik çalışır.

```
CREATE TABLE IF NOT EXISTS orders (
  order_id SERIAL PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
  order_date DATE NOT NULL
);
CREATE TABLE IF NOT EXISTS daily_sales_summary (
  sale_date DATE PRIMARY KEY,
  total_revenue NUMERIC(14,2) NOT NULL,
  total_quantity INTEGER NOT NULL,
  order_count INTEGER NOT NULL,
  average_order_value NUMERIC(14,2) NOT NULL
);
INSERT INTO orders (product_name, quantity, unit_price, order_date)
VALUES
('Laptop',2,25000,'2026-10-20'),
('Mouse',5,500,'2026-10-20'),
('Klavye',3,1500,'2026-10-21'),
('Monitor',2,8000,'2026-10-21'),
('Kulaklik',4,1200,'2026-10-22'),
('Laptop',1,25000,'2026-10-22'),
('Mouse',8,500,'2026-10-23'),
('Monitor',1,8000,'2026-10-23'),
('Klavye',6,1500,'2026-10-24'),
('Kulaklik',3,1200,'2026-10-24');
```

Mevcut veritabanında orders doluysa init.sql tekrar uygulanmamalıdır: aynı siparişler mükerrer eklenebilir. Önce SELECT COUNT(*) FROM orders ile kontrol edin.

6. Replica başlangıç betiği

postgres/replica-entrypoint.sh dosyasını oluşturun. İlk açılışta Primary'den pg_basebackup alır ve -R sayesinde standby.signal ile primary_conninfo yapılandırmasını oluşturur. Sonraki açılışlarda mevcut Replica verisini kullanır.

```
#!/bin/bash
set -euo pipefail

if [ ! -s "$PGDATA/PG_VERSION" ]; then
  echo "Replica: Primary'den base backup aliniyor..."
  mkdir -p "$PGDATA"
  chown -R postgres:postgres "$PGDATA"
  chmod 700 "$PGDATA"
  export PGPASSWORD="$REPLICATION_PASSWORD"
  gosu postgres pg_basebackup \
    -h postgres-primary -p 5432 -U replicator \
    -D "$PGDATA" -Fp -Xs -P -R
  unset PGPASSWORD
fi
exec docker-entrypoint.sh postgres
```

```
chmod +x postgres/replica-entrypoint.sh
```

Not: Replica, Primary'de replikasyon kullanıcısı ve pg_hba.conf izni oluşturulduktan sonra başlatılacaktır. Base backup sırasında Primary ve Replica aynı PostgreSQL major sürümünü (17) kullanmalıdır.

7. Docker Compose: Primary, Replica, Python, Grafana

docker-compose.yml dosyasının tamamı:

```
services:
  postgres-primary:
    image: postgres:17
    container_name: ecommerce-primary
    restart: unless-stopped
    environment:
      POSTGRES_USER: ecommerce_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ecommerce_db
    command: >
      postgres -c wal_level=replica
               -c max_wal_senders=10
               -c max_replication_slots=10
               -c hot_standby=on
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecommerce_user -d ecommerce_db"]
      interval: 5s
      timeout: 5s
      retries: 15

  postgres-replica:
    image: postgres:17
    container_name: ecommerce-replica
    restart: unless-stopped
    profiles: ["replica"]
    environment:
      PGDATA: /var/lib/postgresql/data
      REPLICATION_PASSWORD: ${REPLICATION_PASSWORD}
    entrypoint: ["/bin/bash", "/usr/local/bin/replica-entrypoint.sh"]
    ports:
      - "127.0.0.1:5433:5432"
    volumes:
      - replica_data:/var/lib/postgresql/data
```

```
      - ./postgres/replica-entrypoint.sh:/usr/local/bin/replica-entrypoint.sh:ro
    depends_on:
      postgres-primary:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecommerce_user -d ecommerce_db"]
      interval: 5s
      timeout: 5s
      retries: 20

  python:
    build: ./python
    profiles: ["job"]
    environment:
      DB_READ_HOST: postgres-replica
      DB_WRITE_HOST: postgres-primary
      DB_NAME: ecommerce_db
      DB_USER: ecommerce_user
      DB_PASSWORD: ${POSTGRES_PASSWORD}
    restart: "no"

  grafana:
    image: grafana/grafana:12.2.0
    container_name: ecommerce-grafana
    restart: unless-stopped
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD}
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana

volumes:
  postgres_data:
  replica_data:
  grafana_data:
```

Profiles sayesinde ilk önce yalnızca Primary başlatılabilir. Python, Replica hazır olduktan sonra tek seferlik job olarak çalıştırılır. Servisler Compose'un varsayılan ağı üzerinden birbirlerini isimleriyle bulur.

8. Primary başlatma ve replikasyon kullanıcısı

```
sudo docker compose config
sudo docker compose up -d postgres-primary
sudo docker compose ps
sudo docker exec ecommerce-primary pg_isready \
  -U ecommerce_user -d ecommerce_db
```

Primary'de replicator rolünü oluşturun. Aşağıdaki komut .env dosyasındaki şifreyi kullanır; shell geçmişinde açık şifre yazmaktan kaçınmak için değişkeni dosyadan okur.

```
set -a; source .env; set +a
sudo docker exec -e PGPASSWORD="$POSTGRES_PASSWORD" \
  ecommerce-primary psql -U ecommerce_user -d ecommerce_db \
  -v ON_ERROR_STOP=1 \
  -v repl_pass="$REPLICATION_PASSWORD" \
  -c "SELECT 1;"
```

Rol oluşturmak için aşağıdaki interaktif SQL yöntemini kullanın; parola kısmına .env içindeki REPLICATION_PASSWORD değerini yazın:

```
sudo docker exec -it ecommerce-primary \
  psql -U ecommerce_user -d ecommerce_db

CREATE ROLE replicator WITH REPLICATION LOGIN
  PASSWORD 'replica123_change_me';
\q
```

Parolayı .env'de değiştirdiyseniz SQL'de de aynı değeri kullanın. Mevcut rol varsa CREATE ROLE yerine ALTER ROLE replicator PASSWORD ... komutunu kullanın.

pg_hba.conf erişim kuralı

Yalnızca eğitim amaçlı Compose ağında replication kullanıcısına TCP bağlantısı açıyoruz. Production ortamında IP aralığını kesin sınırlandırın, TLS ve secret yönetimi uygulayın.

```
sudo docker exec ecommerce-primary bash -c \
  "echo 'host replication replicator 0.0.0.0/0 scram-sha-256' \
  >> "\$PGDATA/pg_hba.conf""
sudo docker exec ecommerce-primary psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT pg_reload_conf();"
```

Bu kuralı bir kez eklemek yeterlidir. Primary'nin mevcut veri dizininde eski kurallar varsa pg_hba.conf sırasını inceleyin; PostgreSQL ilk eşleşen kuralı uygular.

9. Replica oluşturma ve doğrulama

```
sudo docker compose --profile replica up -d postgres-replica
sudo docker compose --profile replica ps
sudo docker compose --profile replica logs -f postgres-replica
```

Replica ilk kez açılırken pg_basebackup tamamlanmasını bekleyin. Ardından aşağıdaki testleri yapın.

```
# Primary: bağlı standby var mı?
sudo docker exec ecommerce-primary psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT application_name, state, sync_state
      FROM pg_stat_replication;"

# Replica: gerçekten recovery/standby modunda mı?
sudo docker exec ecommerce-replica psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT pg_is_in_recovery();"

# Replica'da siparişler var mı?
sudo docker exec ecommerce-replica psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT COUNT(*) FROM orders;"
```

Beklenen: Primary üzerinde pg_stat_replication sorgusunda state=streaming; Replica üzerinde pg_is_in_recovery()=t; orders kayıt sayısı başlangıç verisinde 10.

Salt okunur testi

```
sudo docker exec ecommerce-replica psql \
  -U ecommerce_user -d ecommerce_db \
  -c "INSERT INTO orders(product_name,quantity,unit_price,order_date)
      VALUES('Test',1,1,CURRENT_DATE);"
```

Beklenen: cannot execute INSERT in a read-only transaction benzeri hata. Bu testin başarısız olması istenen davranıştır.

10. Python ETL uygulaması

python/requirements.txt:

```
psycopg2-binary==2.9.10
```

python/Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

python/app.py dosyasının tamamı:

```
import os
from collections import defaultdict
from decimal import Decimal
import psycopg2

COMMON = dict(
    dbname=os.getenv("DB_NAME", "ecommerce_db"),
    user=os.getenv("DB_USER", "ecommerce_user"),
    password=os.environ["DB_PASSWORD"],
    port=5432,
    connect_timeout=10,
)

read_conn = psycopg2.connect(
    host=os.getenv("DB_READ_HOST", "postgres-replica"),
    **COMMON
)
write_conn = psycopg2.connect(
    host=os.getenv("DB_WRITE_HOST", "postgres-primary"),
    **COMMON
)

try:
    # READ: yalnızca Replica
    with read_conn.cursor() as cur:
        cur.execute("""
            SELECT order_id, quantity, unit_price, order_date
            FROM orders ORDER BY order_id
        """)
        orders = cur.fetchall()

    daily = defaultdict(lambda: {
        "revenue": Decimal("0"),
        "quantity": 0,
        "orders": 0,
    })
    for _id, quantity, price, date in orders:
        daily[date]["revenue"] += quantity * price
        daily[date]["quantity"] += quantity
        daily[date]["orders"] += 1

    # WRITE: yalnızca Primary, tek transaction
    with write_conn:
        with write_conn.cursor() as cur:
            cur.execute("DELETE FROM daily_sales_summary")
            for date, values in sorted(daily.items()):
                avg = (values["revenue"] / values["orders"]).quantize(
                    Decimal("0.01")
                )
                cur.execute("""
                    INSERT INTO daily_sales_summary
                      (sale_date, total_revenue, total_quantity,
                       order_count, average_order_value)
                    VALUES (%s, %s, %s, %s, %s)
                """, (
                    date, values["revenue"], values["quantity"],
                    values["orders"], avg
                ))

    print(f"{len(orders)} sipariş okundu.")
    print(f"{len(daily)} günlük özet Primary'ye yazıldı.")
finally:
    read_conn.close()
    write_conn.close()
```

Tutarlılık notu: Asenkron Replica gecikebilir. Yeni sipariş eklendikten hemen sonra Python'u çalıştırmak eksik veri okumasına neden olabilir. Testlerde Replica'da yeni kaydın göründüğünü doğruladıktan sonra job'u çalıştırın. Gerçek üretimde WAL replay konumu veya uygulama checkpoint'iyle senkronizasyon bekleme mekanizması eklenmelidir.

11. Python job çalıştırma ve sonuçları inceleme

```
sudo docker compose build python
sudo docker compose --profile replica --profile job \
  run --rm --no-deps python

sudo docker exec ecommerce-primary psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT * FROM daily_sales_summary ORDER BY sale_date;"

sudo docker exec ecommerce-replica psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT * FROM daily_sales_summary ORDER BY sale_date;"
```

Başlangıçta 10 sipariş ve 5 günlük özet beklenir. Toplam ciro 127.400 TL, toplam sipariş sayısı 10'dur. Replica'da sonuçlar hemen görünmezse kısa süre bekleyip tekrar sorgulayın.

12. Grafana kurulumu ve PostgreSQL bağlantısı

```
sudo docker compose up -d grafana
sudo docker compose ps
```

Ubuntu VM içindeki tarayıcıdan http://localhost:3000 açın. Kullanıcı: admin, parola: .env içindeki GRAFANA_ADMIN_PASSWORD. Connections > Data sources > Add data source > PostgreSQL:

Ayar	Değer
Host	postgres-replica:5432
Database	ecommerce_db
Username	ecommerce_user
Password	.env içindeki POSTGRES_PASSWORD
TLS/SSL Mode	disable (yalnızca yerel Docker ağı)
PostgreSQL version	17

Save & test ile bağlantıyı doğrulayın. Grafana'ya üretimde ayrı, yalnızca SELECT yetkili bir rol verilmesi daha güvenlidir; Replica'nın fiziksel read-only olması tek başına en az yetki ilkesinin yerine geçmez.

13. Grafana dashboard: dört panel

Dashboards > New > New dashboard > Add visualization. PostgreSQL veri kaynağını seçin; SQL Code mode kullanın. Dashboard tarih aralığını 2026-10-20 ile 2026-10-25 arasına ayarlayın.

Panel 1 - Günlük ciro (Time series)

```
SELECT sale_date AS "time", total_revenue AS "Günlük Ciro"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date)
ORDER BY sale_date;
```

Panel 2 - Toplam ciro (Stat)

```
SELECT COALESCE(SUM(total_revenue),0) AS "Toplam Ciro"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date);
```

Panel 3 - Sipariş sayısı (Stat)

```
SELECT COALESCE(SUM(order_count),0) AS "Sipariş Sayısı"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date);
```

Panel 4 - Günlük satış tablosu (Table)

```
SELECT sale_date AS "Tarih", total_revenue AS "Ciro",
       total_quantity AS "Ürün Adedi",
       order_count AS "Sipariş Sayısı",
       average_order_value AS "Ortalama Sipariş"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date)
ORDER BY sale_date;
```

Stat panelleri için uygun birim seçin (TRY); dashboard'u kaydedin. Grafana sadece Replica'ya SELECT sorguları gönderir.

14. Uçtan uca değişiklik testi

Primary'ye 24 Ekim tarihli 3 adet, birim fiyatı 30.000 TL olan yeni Laptop siparişi ekleyin:

```
sudo docker exec ecommerce-primary psql \
  -U ecommerce_user -d ecommerce_db \
  -c "INSERT INTO orders(product_name,quantity,unit_price,order_date)
      VALUES ('Laptop',3,30000,'2026-10-24');"
```

Replica'ya ulaşmasını doğrulayın:

```
sudo docker exec ecommerce-replica psql \
  -U ecommerce_user -d ecommerce_db \
  -c "SELECT COUNT(*) FROM orders;"
```

11 kayıt göründükten sonra Python job'u tekrar çalıştırın:

```
sudo docker compose --profile replica --profile job \
  run --rm --no-deps python
```

24 Ekim için beklenen: 102.600 TL ciro, 12 ürün, 3 sipariş, 34.200 TL ortalama sipariş tutarı. Grafana'yı yenileyerek güncellemeyi doğrulayın.

15. Replikasyon gecikmesi ve gözlemleme

Primary'de:

```
SELECT application_name, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM pg_stat_replication;
```

Replica'da:

```
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(),
       pg_last_wal_replay_lsn(),
       now() - pg_last_xact_replay_timestamp() AS replay_age;
```

replay_age uzun süre yeni transaction yoksa da büyüyebilir; tek başına kesin replikasyon gecikmesi göstergesi değildir. Yeni veri yazıp Replica'da görünmesini doğrulamak en anlaşılır fonksiyonel testtir.

16. Sık görülen sorunlar

Belirti	Kontrol / çözüm
Replica pg_basebackup bağlanamıyor	Primary rolü, pg_hba.conf, parola ve Docker DNS adını kontrol edin.
Replica read-only değil	pg_is_in_recovery() sorgusunun t döndürdüğünü doğrulayın.
Grafana bağlantı hatası	Host postgres-replica:5432 olmalı; localhost:5433 değil.
Grafana grafik boş	Zaman aralığını 20-25 Ekim 2026 seçin.
Python connection refused	Replica healthy ve Compose ağında erişilebilir olmalı.
İlk veri yok	Primary init.sql yalnızca boş volume ile ilk kez çalışır.
Port already allocated	Host 5432 veya 5433 başka servis tarafından kullanılıyor.
Replica sürekli yeniden başlıyor	sudo docker compose --profile replica logs postgres-replica.

17. Operasyon ve güvenlik notları

Servisleri durdurmak için sudo docker compose --profile replica stop, kaldırmak için sudo docker compose --profile replica down kullanabilirsiniz. Volume'ları silen -v parametresini yedeksiz kullanmayın. Replica Primary'nin yedeğinin yerini tutmaz; yanlışlıkla silinen kayıtlar da replike edilir. Primary arızalanırsa bu örnek mimari otomatik failover yapmaz.

Bu laboratuvarda tek VM kullanıldığı için CPU/RAM/disk tamamen ayrışmaz. Gerçek kapasite ve dayanıklılık için ayrı host'lar, ayrı depolama, gözlemleme, replikasyon slotu/WAL saklama politikası, TLS, kısıtlı pg_hba ağları, ayrı read-only kullanıcı ve failover tasarımı gerekir.

18. Tamamlanma kontrol listesi

☐ Primary sağlıklı; ☐ Replica streaming; ☐ pg_is_in_recovery() = true; ☐ Replica INSERT reddediliyor; ☐ Python SELECT Replica'dan; ☐ Python WRITE Primary'ye; ☐ Özetler replike ediliyor; ☐ Grafana Replica'ya bağlı; ☐ Yeni sipariş testinde paneller güncelleniyor.

Kaynaklar: PostgreSQL 17 belgeleri: Warm Standby / Streaming Replication, Hot Standby, pg_basebackup, pg_stat_replication; Docker Official postgres image belgeleri; Grafana PostgreSQL data source belgeleri. Resmî başvuru adresleri: postgresql.org/docs/17/ , hub.docker.com/_/postgres , grafana.com/docs/grafana/latest/datasources/postgres/